

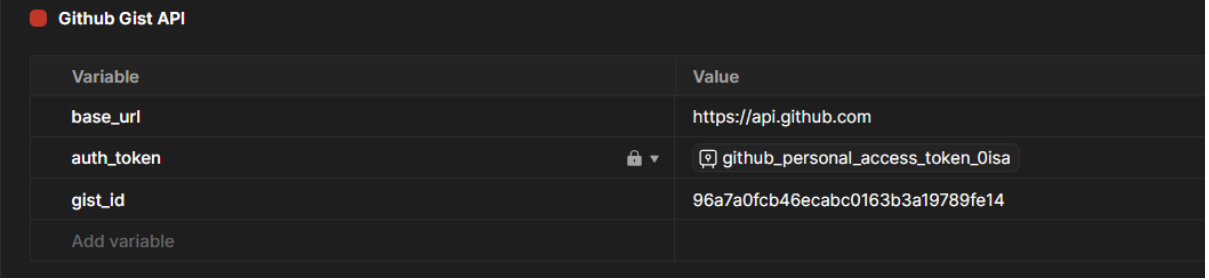
Exploring and Testing APIs with Postman

GitHub Repository: https://github.com/Elizabeth-Ater/API_Testing

Postman Environment Setup

Explanation

The Postman environment contains the variables required for the API requests. These variables allow the same values to be reused across different requests and make the collection easier to manage.



A screenshot of the Postman 'Environment' tab for a collection named 'Github Gist API'. It shows a table with two columns: 'Variable' and 'Value'. The variables listed are 'base_url' with value 'https://api.github.com', 'auth_token' with value 'github_personal_access_token_0lsa' (masked with a square icon), and 'gist_id' with value '96a7a0fcb46ecabc0163b3a19789fe14'. There is an 'Add variable' button at the bottom.

Variable	Value
base_url	https://api.github.com
auth_token	github_personal_access_token_0lsa
gist_id	96a7a0fcb46ecabc0163b3a19789fe14
Add variable	

Pre-Request Script

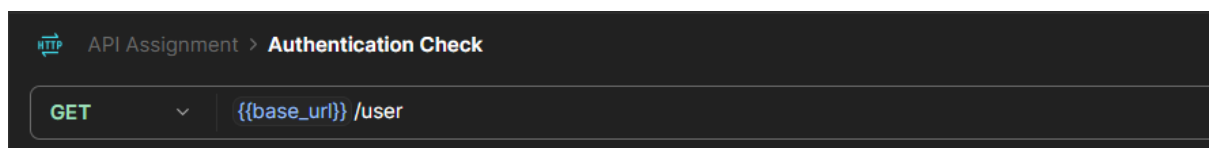
This project did not use a pre-request script, since GitHub Personal Access Tokens do not require refreshing during a session like OAuth2 tokens do. Although the token can expire, this requires manually generating a new one in GitHub's developer settings, not an automatic refresh. The token is stored and reused for all requests through Bearer Token authorization.

Authentication Check

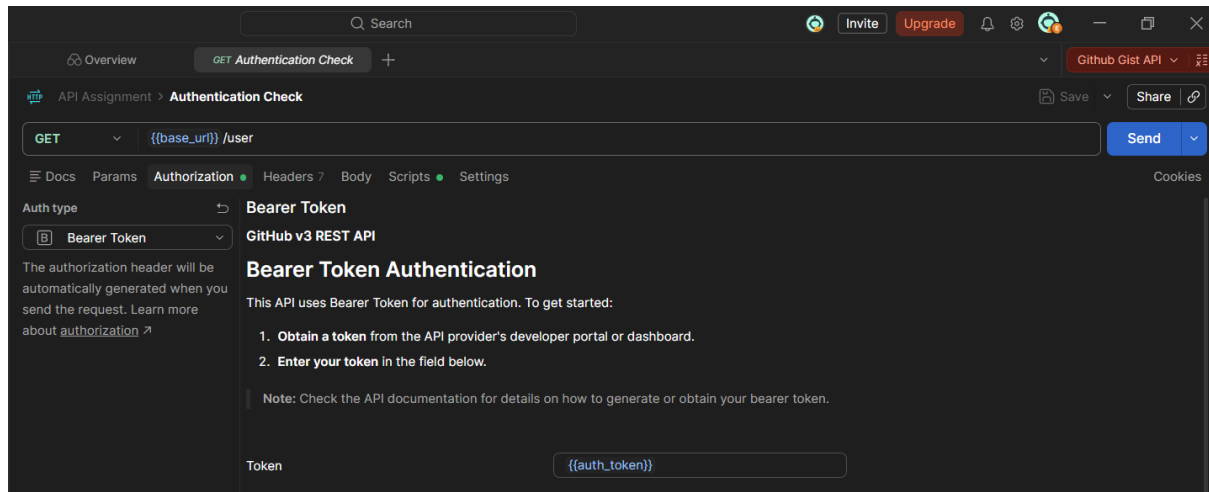
Explanation

This request checks whether the GitHub Personal Access Token is valid. A successful response confirms that the request is authenticated and that an authenticated GitHub user exists.

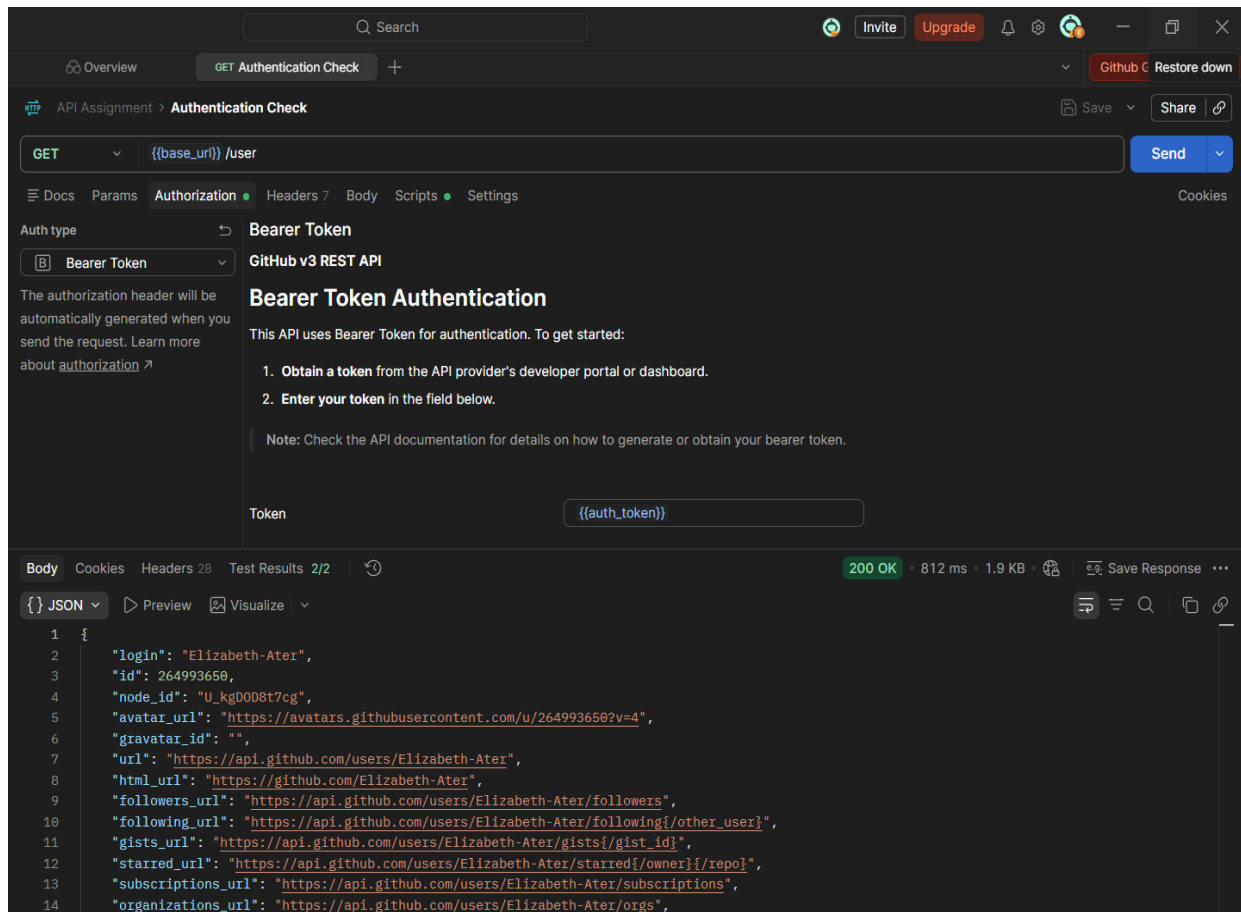
Method and Url: GET



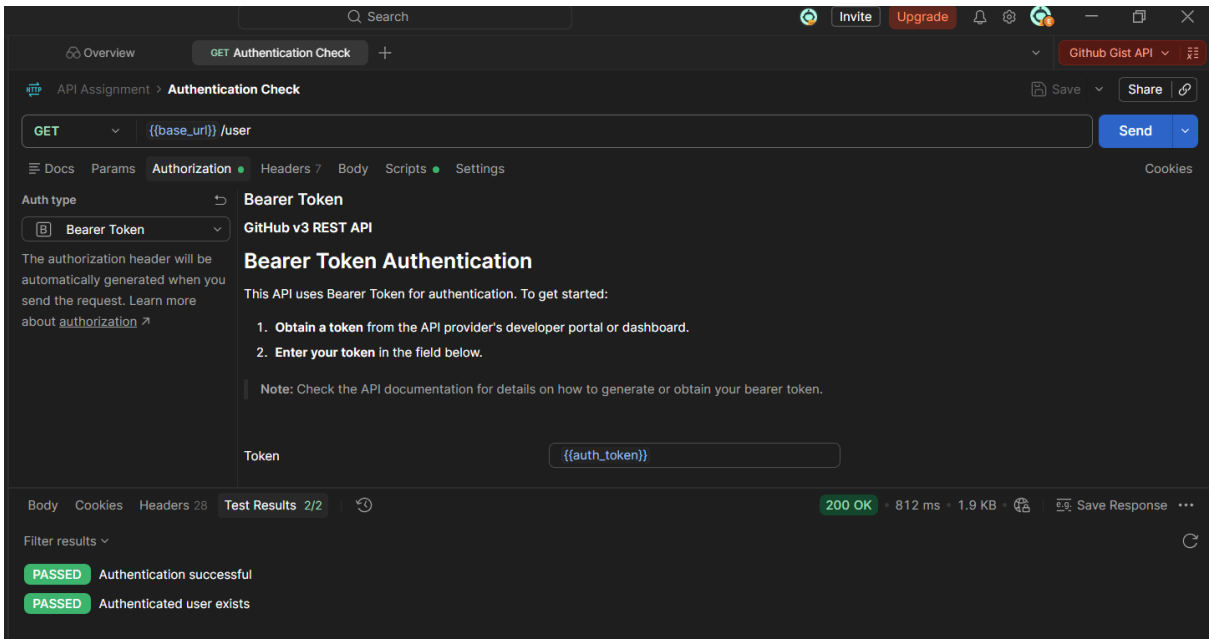
Authorization



Response



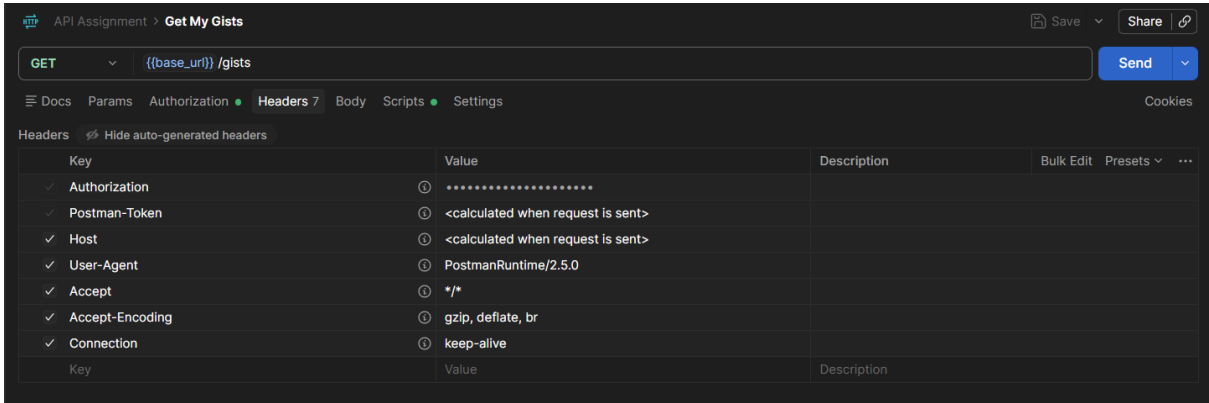
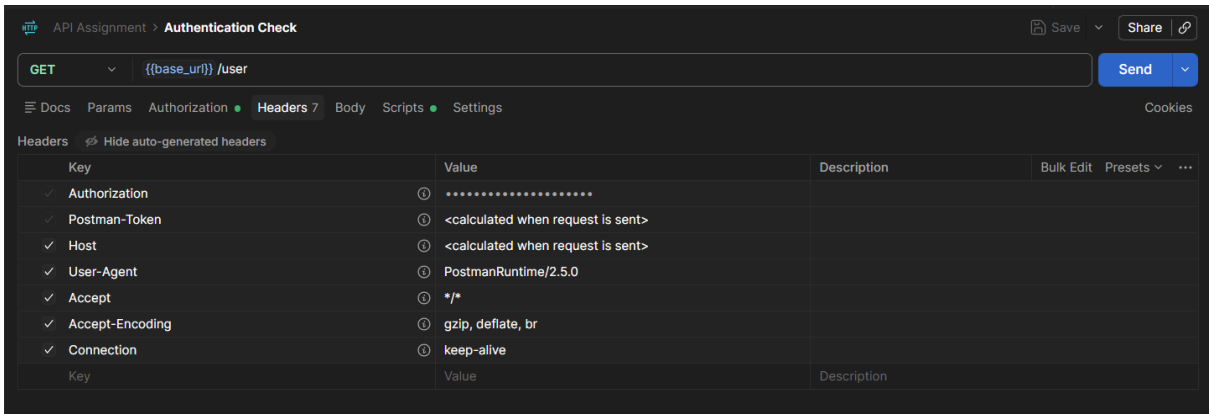
Test



Headers

Explanation

This section shows the Headers tab for each request. All requests include the Authorization header, which is generated automatically from the Bearer Token. The Create a Gist and Update a Gist requests also include a Content-Type header, since they send a JSON body.



API Assignment > Create Gist

POST{{base_url}} /gists

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	①				
✓ Postman-Token	① <calculated when request is sent>				
✓ Content-Type	① application/json				
✓ Content-Length	① <calculated when request is sent>				
✓ Host	① <calculated when request is sent>				
✓ User-Agent	① PostmanRuntime/2.5.0				
✓ Accept	① */*				
✓ Accept-Encoding	① gzip, deflate, br				
✓ Connection	① keep-alive				
Key	Value	Description			

API Assignment > Get Single Gist

GET{{base_url}} /gists/ {{gist_id}}

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Headers Hide auto-generated headers

Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	①				
✓ Postman-Token	① <calculated when request is sent>				
✓ Host	① <calculated when request is sent>				
✓ User-Agent	① PostmanRuntime/2.5.0				
✓ Accept	① */*				
✓ Accept-Encoding	① gzip, deflate, br				
✓ Connection	① keep-alive				
Key	Value	Description			

API Assignment > Update Gist

PATCH{{base_url}} /gists/ {{gist_id}}

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	①				
✓ Postman-Token	① <calculated when request is sent>				Go to settings
✓ Content-Type	① application/json				
✓ Content-Length	① <calculated when request is sent>				
✓ Host	① <calculated when request is sent>				
✓ User-Agent	① PostmanRuntime/2.5.0				
✓ Accept	① */*				
✓ Accept-Encoding	① gzip, deflate, br				
✓ Connection	① keep-alive				
Key	Value	Description			

API Assignment > Delete Gist

DELETE{{base_url}} /gists/ {{gist_id}}

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Headers Hide auto-generated headers

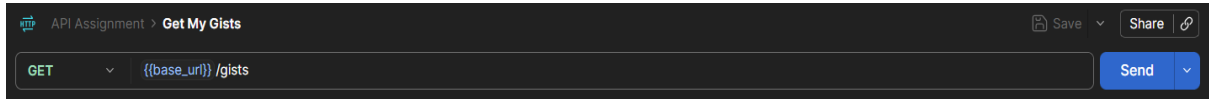
Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	①				
✓ Postman-Token	① <calculated when request is sent>				
✓ Host	① <calculated when request is sent>				
✓ User-Agent	① PostmanRuntime/2.5.0				
✓ Accept	① */*				
✓ Accept-Encoding	① gzip, deflate, br				
✓ Connection	① keep-alive				
Key	Value	Description			

Get My Gists

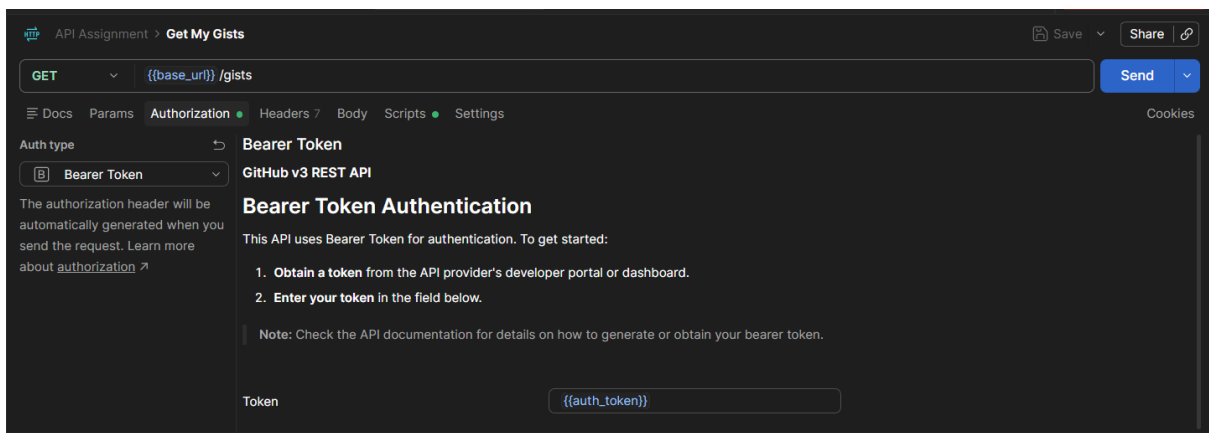
Explanation

This request retrieves the authenticated user's Gists from GitHub. A successful response confirms that the request was processed correctly.

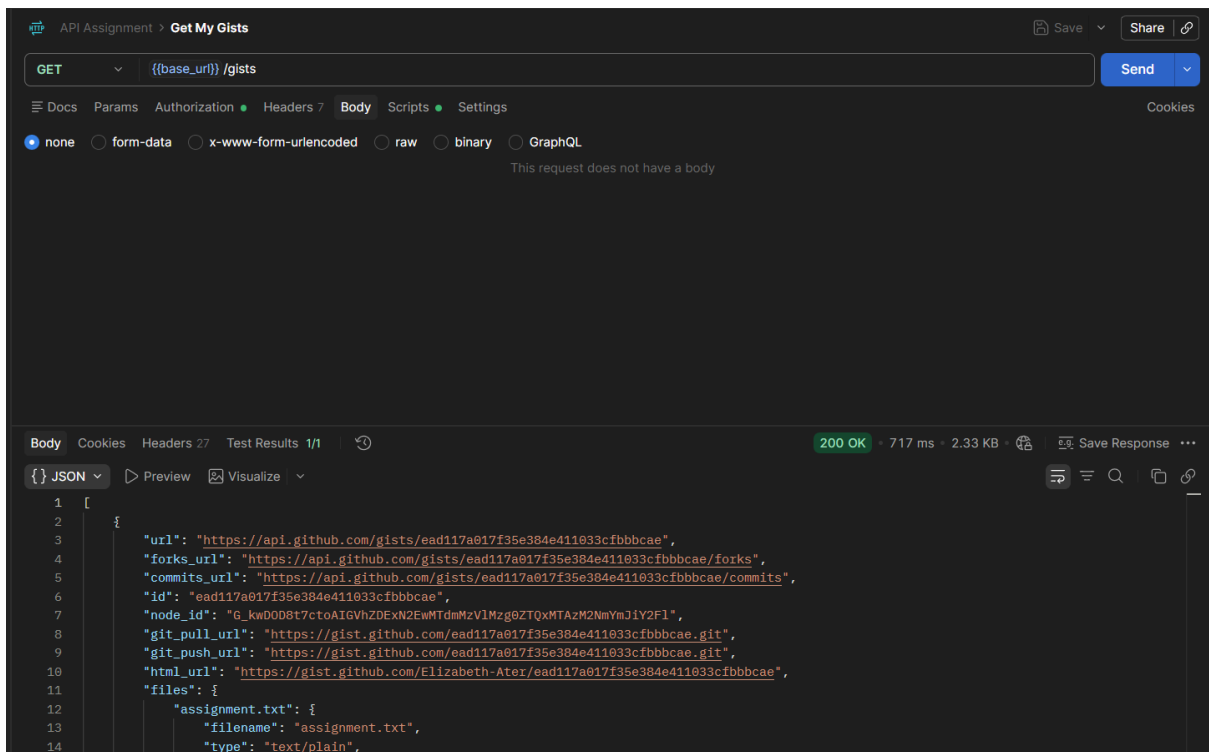
Method and Url: GET



Authorization



Response



Test

The screenshot shows the REST Client interface for an API assignment titled "Get My Gists". The request method is GET and the URL is {{base_url}} /gists. The authentication type is Bearer Token. The test results show a 200 OK status with a response time of 608 ms and a body size of 2.09 KB. The status code is 200 and the response is PASSED.

Create a Gist

Explanation

This request creates a new Gist. The response provides the Gist ID, which is stored in the `gist_id` environment variable for use in later requests.

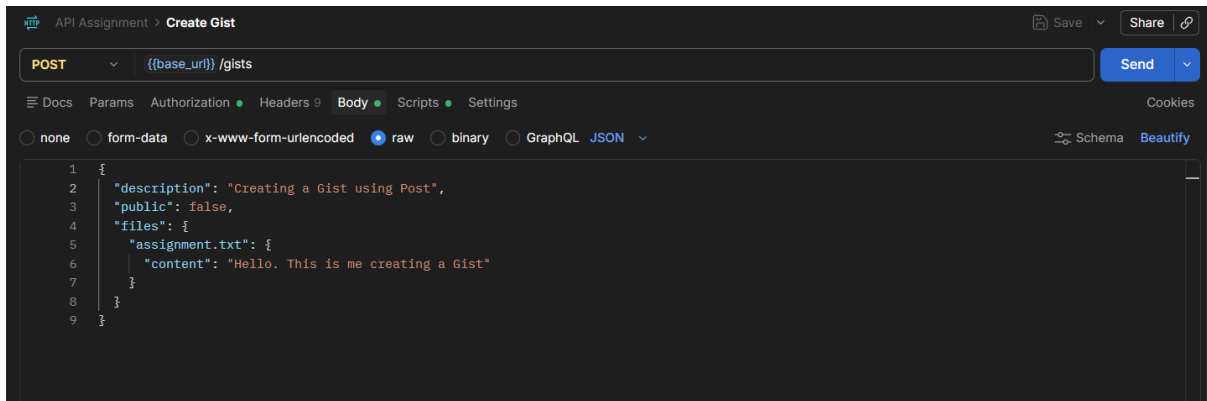
Method and Url: POST

The screenshot shows the REST Client interface for an API assignment titled "Create Gist". The request method is POST and the URL is {{base_url}} /gists.

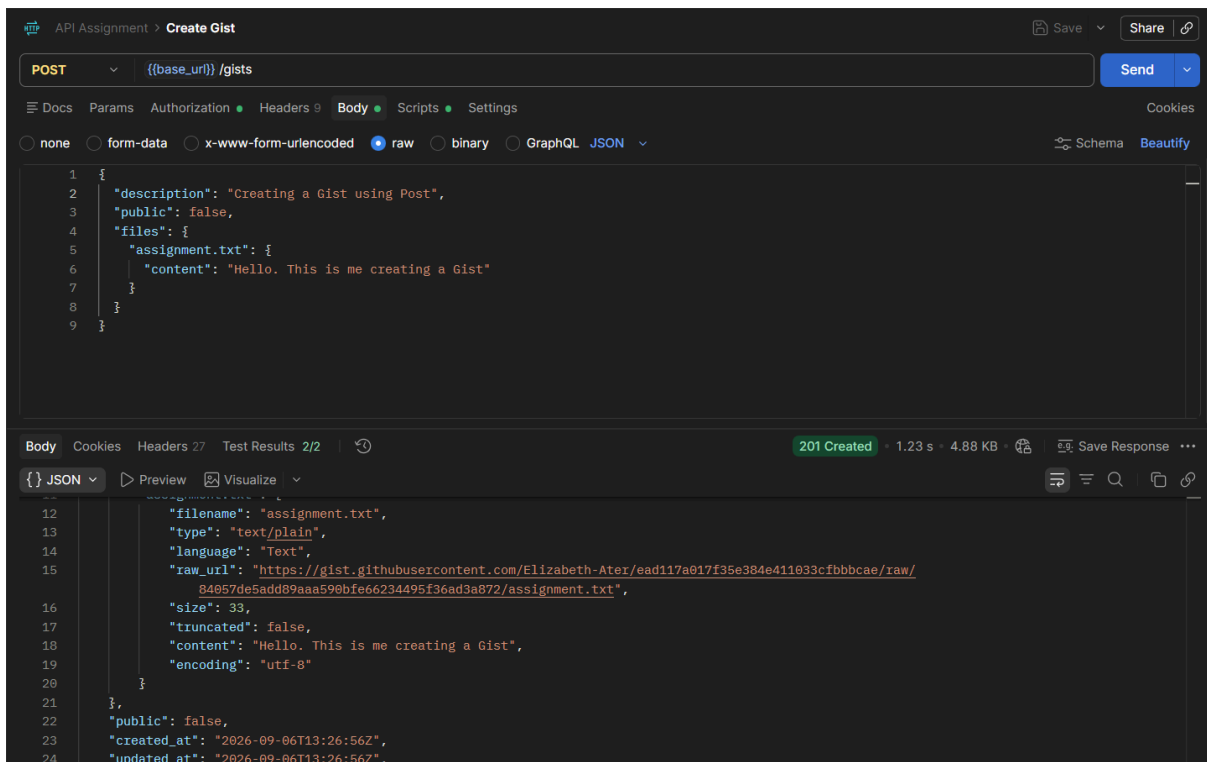
Authorization

The screenshot shows the REST Client interface for an API assignment titled "Create Gist". The request method is POST and the URL is {{base_url}} /gists. The authentication type is Bearer Token. The interface shows the steps to obtain a token and enter it in the field below. The token field contains {{auth_token}}.

Body



Response



Test

The screenshot shows the Postman interface for a POST request to the GitHub Gists API. The URL is `{{base_url}} /gists`. The request body is a JSON object:

```
{  "description": "Postman API Assignment",  "public": false,  "files": {    "assignment.txt": {      "content": "This Gist was created using the GitHub API and Postman."    }  }}
```

. The response status is 201 Created, with a time of 1.37 s and a size of 4.89 KB. The test results show two passed assertions: "Status code is 201" and "Gist has an ID".

Get a Single Gist

Explanation

This request retrieves a specific Gist using the `gist_id` stored in the Postman environment.

Method and Url: GET

The screenshot shows the Postman interface for a GET request to the GitHub Gists API. The URL is `{{base_url}} /gists/ {{gist_id}}`. The request method is GET.

Authorization

The screenshot shows the Postman interface for a GET request to the GitHub Gists API, with the Authorization tab selected. The Auth type is Bearer Token. The token field contains `{{auth_token}}`. The interface also displays instructions for Bearer Token Authentication, including steps to obtain a token and enter it in the field.

Response

API Assignment > Get Single Gist

SaveShare

GET{{(base_url)}} /gists/ {{gist_id}}

Send

DocsParamsAuthorizationHeaders 7BodyScriptsSettingsCookies

Auth type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Bearer Token

GitHub v3 REST API

Bearer Token Authentication

This API uses Bearer Token for authentication. To get started:

- Obtain a token from the API provider's developer portal or dashboard.
- Enter your token in the field below.

Note: Check the API documentation for details on how to generate or obtain your bearer token.

Token{{auth_token}}

BodyCookiesHeaders 28Test Results 2/2

200 OK669 ms2.32 KBSave Response

JSON

PreviewVisualize

```
1 {
2   "url": "https://api.github.com/gists/dd874121a25aa741aa622b51435d8237",
3   "forks_url": "https://api.github.com/gists/dd874121a25aa741aa622b51435d8237/forks",
4   "commits_url": "https://api.github.com/gists/dd874121a25aa741aa622b51435d8237/commits",
5   "id": "dd874121a25aa741aa622b51435d8237",
6   "node_id": "G_kwDOD8t7ctoAIGRKODc0MTIxYTIIYWE3NDZhYTtyMmI1MTQzNWQ4MjM3",
7   "git_pull_url": "https://gist.github.com/dd874121a25aa741aa622b51435d8237.git",
8   "git_push_url": "https://gist.github.com/dd874121a25aa741aa622b51435d8237.git",
9   "html_url": "https://gist.github.com/Elizabeth-Ater/dd874121a25aa741aa622b51435d8237",
10  "files": {
11    "assignment.txt": {
12      "filename": "assignment.txt",
13      "type": "text/plain",
14      "language": "Text",
```

Test

API Assignment > Get Single Gist

SaveShare

GET{{(base_url)}} /gists/ {{gist_id}}

Send

DocsParamsAuthorizationHeaders 7BodyScriptsSettingsCookies

Auth type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Bearer Token

GitHub v3 REST API

Bearer Token Authentication

This API uses Bearer Token for authentication. To get started:

- Obtain a token from the API provider's developer portal or dashboard.
- Enter your token in the field below.

Note: Check the API documentation for details on how to generate or obtain your bearer token.

Token{{auth_token}}

BodyCookiesHeaders 28Test Results 2/2

200 OK669 ms2.32 KBSave Response

Filter results

PASSEDStatus code is 200

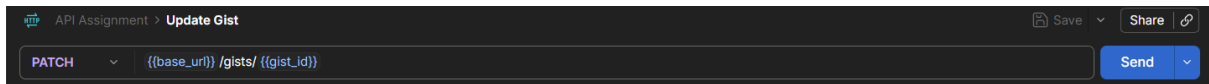
PASSEDGist has an ID

Update a Gist

Explanation

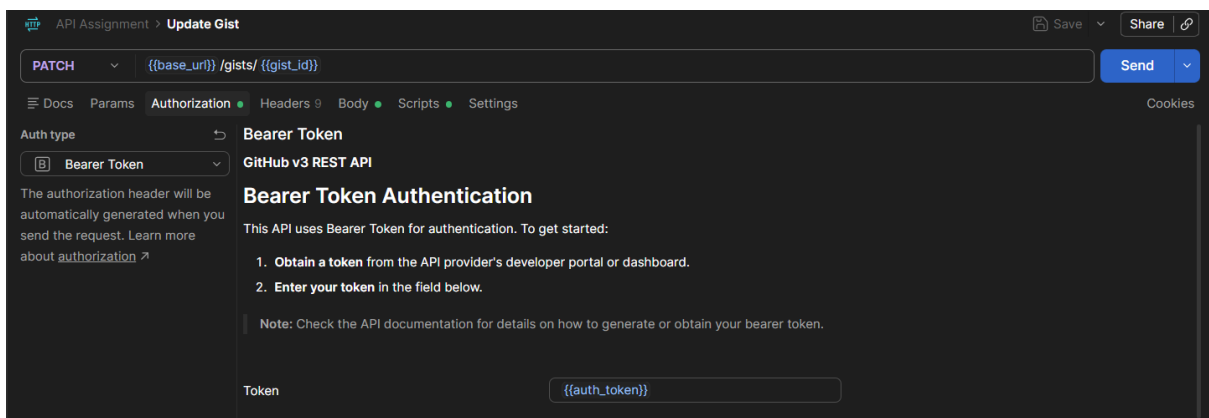
This request updates an existing Gist. The `gist_id` variable identifies the Gist that is being modified.

Method and Url: PATCH



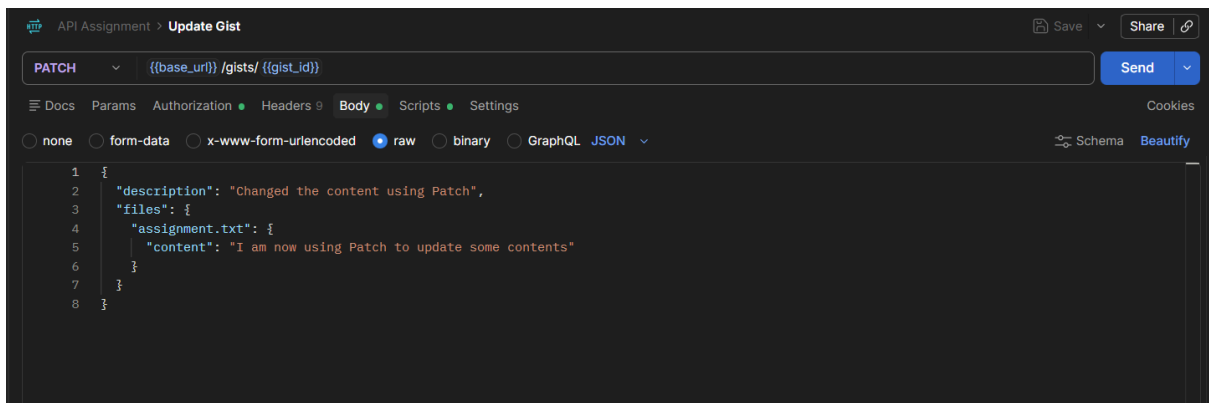
The screenshot shows the top section of an API client interface. At the top, it says "API Assignment > Update Gist". On the right, there are "Save" and "Share" buttons. Below this, there is a dropdown menu set to "PATCH" and a text input field containing the URL template `{{base_url}}/gists/{{gist_id}}`. To the right of the input field is a blue "Send" button with a dropdown arrow.

Authorization



This screenshot shows the "Authorization" tab in the API client. The "Auth type" is set to "Bearer Token". Below this, it says "The authorization header will be automatically generated when you send the request. Learn more about authorization". To the right, under "Bearer Token Authentication", it says "GitHub v3 REST API" and "This API uses Bearer Token for authentication. To get started:". There are two numbered steps: 1. Obtain a token from the API provider's developer portal or dashboard. 2. Enter your token in the field below. A note says: "Note: Check the API documentation for details on how to generate or obtain your bearer token." At the bottom, there is a "Token" label and a text input field containing `{{auth_token}}`.

Body



This screenshot shows the "Body" tab in the API client. At the top, it says "API Assignment > Update Gist". On the right, there are "Save" and "Share" buttons. Below this, there is a dropdown menu set to "PATCH" and a text input field containing the URL template `{{base_url}}/gists/{{gist_id}}`. To the right of the input field is a blue "Send" button with a dropdown arrow. Below the URL field, there are tabs for "Docs", "Params", "Authorization", "Headers", "Body", "Scripts", and "Settings". The "Body" tab is selected. Below the tabs, there are radio buttons for "none", "form-data", "x-www-form-urlencoded", "raw" (which is selected), "binary", and "GraphQL". To the right of these are "Schema" and "Beautify" buttons. The main area shows a JSON body with the following content:

```
1 {
2   "description": "Changed the content using Patch",
3   "files": {
4     "assignment.txt": {
5       "content": "I am now using Patch to update some contents"
6     }
7   }
8 }
```

Response

POST Create Gist

PATCH Update Gist

+

Github Gist API

⌵

⌵

API Assignment > Update Gist

Save ⌵

Share

PATCH ⌵

{{base_uri}} /gists/ {{gist_id}}

Send ⌵

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON ⌵

Schema

Beautify

1

{

2

"description": "Changed the content using Patch",

3

"files": {

4

"assignment.txt": {

5

"content": "I am now using Patch to update some contents"

6

}

7

}

8

}

Body

Cookies

Headers 27

Test Results 1/2

⌵

200 OK

1.21 s

2.34 KB

⌵

Save Response

⋮

{} JSON ⌵

Preview

Visualize ⌵

9

"html_url": "https://gist.github.com/Elizabeth-Ater/ead117a017f35e384e411033cfbbcae",

10

"files": {

11

"assignment.txt": {

12

"filename": "assignment.txt",

13

"type": "text/plain",

14

"language": "Text",

15

"raw_url": "https://gist.githubusercontent.com/Elizabeth-Ater/ead117a017f35e384e411033cfbbcae/raw/8f718d76e624b33d0bf7ff7e0eb1553da3b97ae2/assignment.txt",

16

"size": 44,

17

"truncated": false,

18

"content": "I am now using Patch to update some contents",

19

"encoding": "utf-8"

20

}

21

},

22

}

⚡ Fix failing Gist description assertion

Globals

Vault

Tools

⌵

⌵

Test

POST Create Gist

PATCH Update Gist

+

Github Gist API

⌵

⌵

API Assignment > Update Gist

Save ⌵

Share

PATCH ⌵

{{base_uri}} /gists/ {{gist_id}}

Send ⌵

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON ⌵

Schema

Beautify

1

{

2

"description": "Updated Postman API Assignment",

3

"files": {

4

"assignment.txt": {

5

"content": "This Gist has been updated using the GitHub API and Postman."

6

}

7

}

8

}

Body

Cookies

Headers 27

Test Results 2/2

⌵

200 OK

1.15 s

2.35 KB

⌵

Save Response

⋮

Filter results ⌵

⌵

PASSED

Status code is 200

PASSED

Gist was updated

Explanation

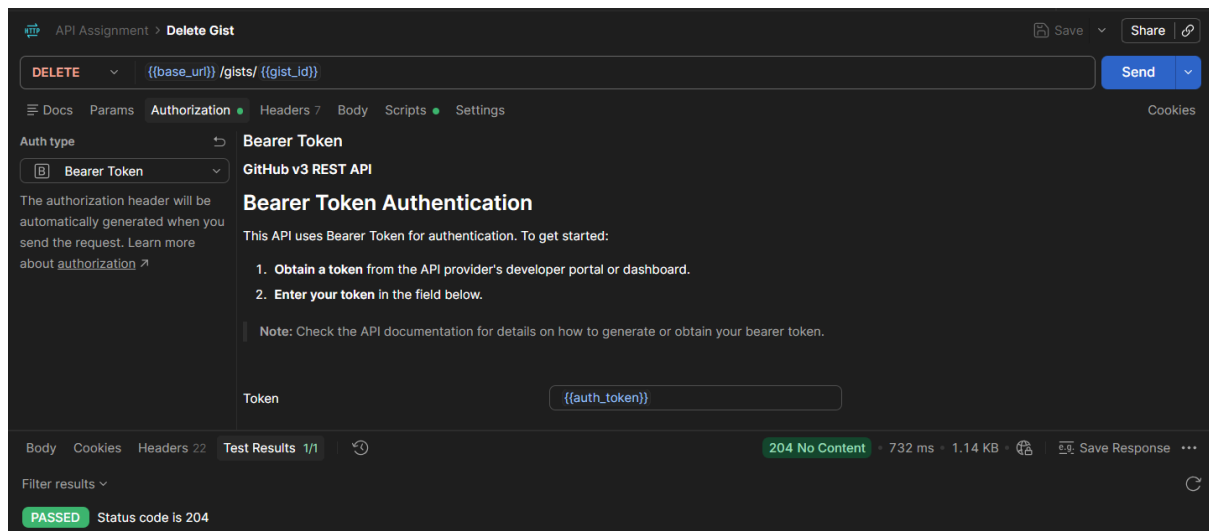
Method and Url: DELETE

Authorization

Response

A screenshot of the GitHub REST API documentation page for Bearer Token authentication. The page is dark-themed. At the top, there's a navigation bar with 'API Assignment' and 'Delete Gist'. Below that, a 'DELETE' method is selected for the endpoint '{{base_url}} /gists/{{gist_id}}'. The 'Auth type' is set to 'Bearer Token'. The main heading is 'Bearer Token Authentication'. It states: 'This API uses Bearer Token for authentication. To get started:'. There are two numbered steps: 1. 'Obtain a token from the API provider's developer portal or dashboard.' and 2. 'Enter your token in the field below.' A note says: 'Note: Check the API documentation for details on how to generate or obtain your bearer token.' Below the note is a 'Token' label and a text input field containing '{{auth_token}}'. At the bottom, there's a status bar showing '204 No Content', '732 ms', '1.14 KB', and a 'Save Response' button. The bottom-most bar shows 'Body', 'Cookies', 'Headers 22', 'Test Results 1/1', and a 'Raw' button.

Test

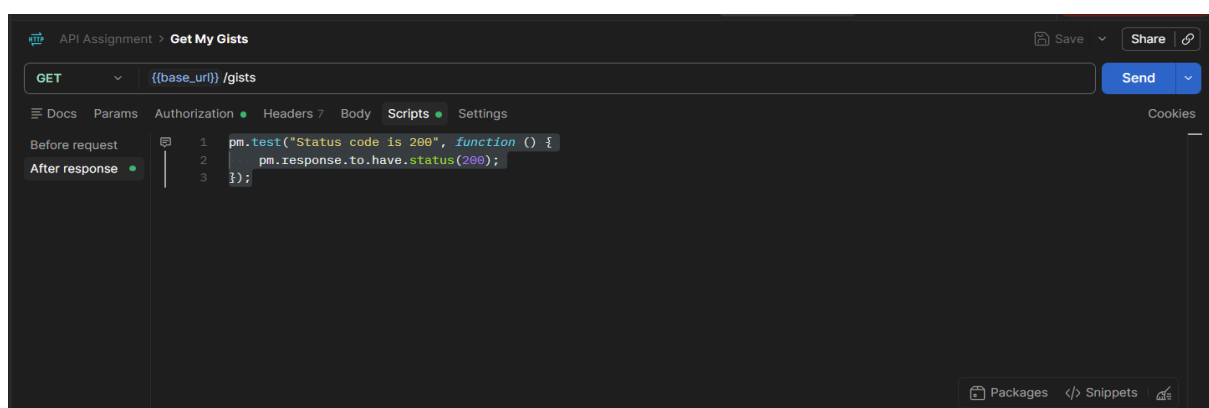
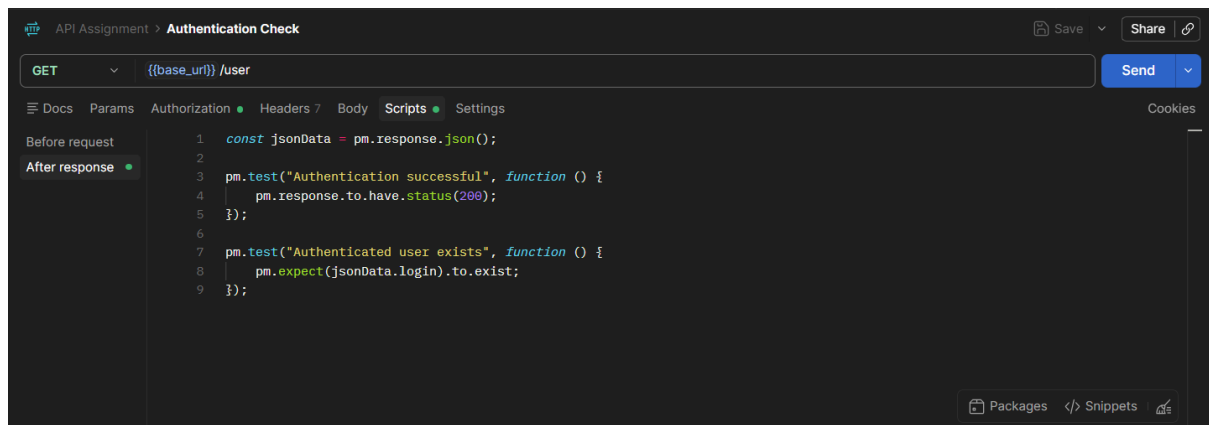


Automated Testing

Explanation

Postman test scripts were used to automatically verify the expected status codes and response data. This makes it possible to confirm that the API requests work correctly without checking every result manually.

Postman Tests



API Assignment > **Create Gist** Save Share

POST `{{base_url}} /gists` Send

Docs Params Authorization Headers 9 Body **Scripts** Settings Cookies

Before request

After response

```
1 const jsonData = pm.response.json();
2
3 pm.environment.set("gist_id", jsonData.id);
4
5 pm.test("Status code is 201", function () {
6   pm.response.to.have.status(201);
7 });
8
9 pm.test("Gist has an ID", function () {
10   pm.expect(jsonData.id).to.exist;
11 });
```

Packages Snippets

API Assignment > **Get Single Gist** Save Share

GET `{{base_url}} /gists/ {{gist_id}}` Send

Docs Params Authorization Headers 7 Body **Scripts** Settings Cookies

Before request

After response

```
1 const jsonData = pm.response.json();
2
3 pm.test("Status code is 200", function () {
4   pm.response.to.have.status(200);
5 });
6
7 pm.test("Gist has an ID", function () {
8   pm.expect(jsonData.id).to.exist;
9 });
```

Packages Snippets

API Assignment > **Update Gist** Save Share

PATCH `{{base_url}} /gists/ {{gist_id}}` Send

Docs Params Authorization Headers 9 Body **Scripts** Settings Cookies

Before request

After response

```
1 const jsonData = pm.response.json();
2
3 pm.test("Status code is 200", function () {
4   pm.response.to.have.status(200);
5 });
6
7 pm.test("Gist was updated", function () {
8   pm.expect(jsonData.description).to.eql("Updated Postman API Assignment");
9 });
```

Packages Snippets

API Assignment > **Delete Gist** Save Share

DELETE `{{base_url}} /gists/ {{gist_id}}` Send

Docs Params Authorization Headers 7 Body **Scripts** Settings Cookies

Before request

After response

```
1 pm.test("Status code is 204", function () {
2   pm.response.to.have.status(204);
3 });
```

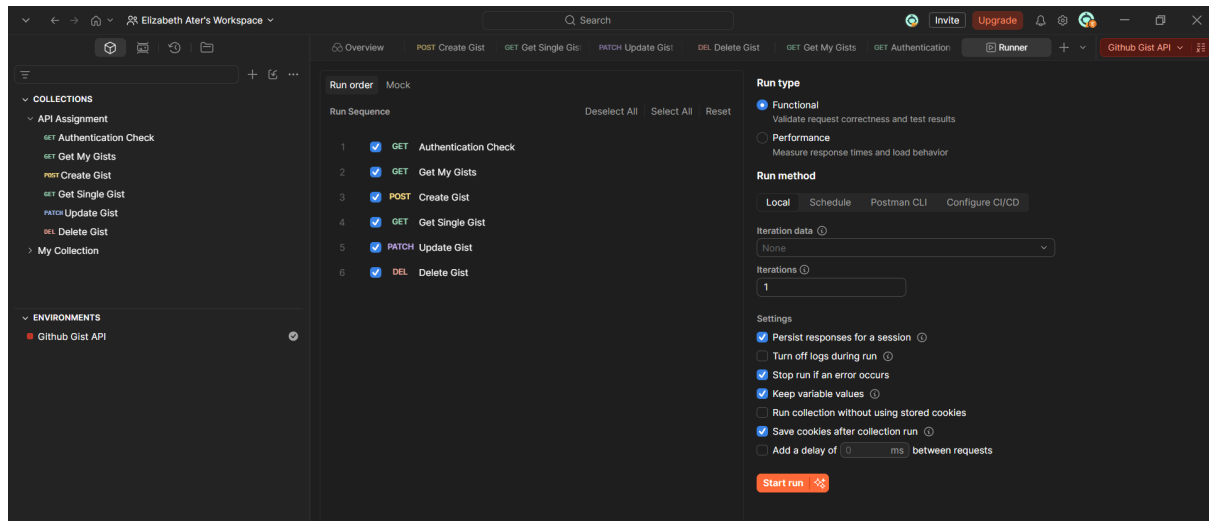
Packages Snippets

Collection Runner

Explanation

The Collection Runner was used to execute the API requests automatically in sequence. The successful results demonstrate that the requests and their automated tests are working correctly.

Collection Runner Setup



Requests Executed

